

AI224: Operating Systems — Lecture 6

Dr. Karim Lounis

Jan - May 2025



Peterson's Algorithm

Peterson's Algorithm

Definition

Peterson's Algo is a solution for the critical section problem involving two processes that alternate their CS execution. It was proposed in 1981.

Recall. The Critical Section Problem is all about designing a protocol that processes can apply to cooperate. Such protocol must satisfy the following three requirements:

- **Mutual Exclusion.** At most, one process is executing its critical section at a time.
- **Progress.** A process outside its critical section should not prevent another process from accessing the critical section.
- **Bounded Waiting.** A process must not wait indefinitely in the queue to access its critical section.

However, no guarantees that it will work correctly on modern architectures.

Peterson's Algorithm

Peterson's Algo

Structure of process P_i

```
do {  
    Interested[i]=true;  
    turn = j;  
    while (Interested[j] && turn==j);  
    CS  
    Interested[i]=false;  
    ...  
} while(true);
```

The variable Interested[] indicates that a process is ready to enter the critical section, whereas the variable turn indicates whose turn it is to enter the critical section.

Peterson's Algorithm

Peterson's Algo

Structure of process P_i

```
do {
    Interested[i]=true;
    turn = j;
    while (Interested[j] && turn==j);
    CS
    Interested[i]=false;
    ...
} while(true);
```

Peterson's algorithms fulfills those three requirements.

But has busy waiting

Hardware solutions for CS

Easy, but not efficient:

Certain OSs running on single-processor systems disable all interrupts when a shared variable is being manipulated (i.e., entering critical region), and then re-enable them just before leaving the critical region.

Early versions of UNIX did that. But it is unwise. Also, disabling interrupt would only affect the CPU that executed the disable instruction.

Modern solution (Cf., Lecture 05):

Certain computers provide a machine-code instruction that performs indivisible read-write operation on some memory location.

e.g., Intel 8086 microprocessor has LOCK instruction prefix to lock access to memory (bus) when executing XCHG instruction

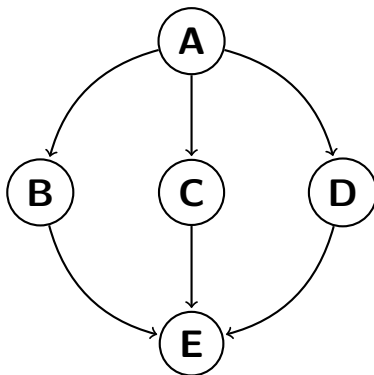
These instructions are real hardware implementations of the abstracted functions `test_and_set()` and `compare_and_swap()`.

PPG for Synchronization

Process Precedence Graph for Synchronization

Recall (Process Precedence Graph)

PPG (Process Precedence Graph): Is a directed graph that is used to graphically express the order on which processes or threads are executed with respect to other processes or threads.



$(A) \rightarrow (B)$ means that A must finish before B can start.

Recall (Process Precedence Graph)

PPG (Process Precedence Graph): Is a directed graph that is used to graphically express the order on which processes are executed with respect to other processes or threads.

Begin

A;

CoBegin;

B, C, D

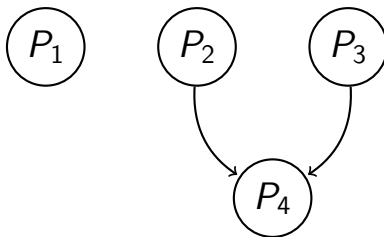
CoEnd;

E;

End

PPG and CoBegin/CoEnd

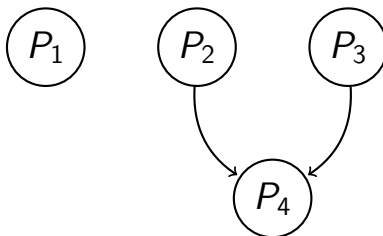
Exercise: Write CoBegin/CoEnd code for the following PPG :



The precedence constraints are: Process P_1 , P_2 , and P_3 should execute in parallel. However, process P_4 should execute after P_2 and P_3 terminate, and can execute before, after, or in parallel with P_1 .

PPG and CoBegin/CoEnd

Exercise: Write CoBegin/CoEnd code for the following PPG :



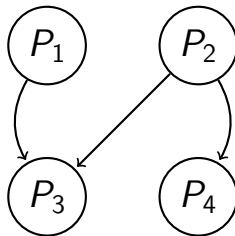
CoBegin
Begin

CoBegin P_2, P_3 **CoEnd**
 P_4 ;

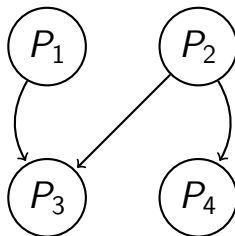
End
 P_1 ;
CoEnd

PPG and CoBegin/CoEnd

Exercise: Write CoBegin/CoEnd code for the following PPG :



PPG and CoBegin/CoEnd



Begin

CoBegin P_1, P_2 ; **CoEnd**

CoBegin P_3, P_4 ; **CoEnd**

End

CoBegin

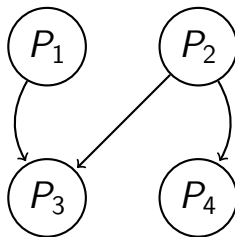
Begin P_2, P_4 ; **End**

Begin P_1, P_3 ; **End**

CoEnd

PPG and CoBegin/CoEnd

Let's enforce a precedence constraint on the PPG:



Begin

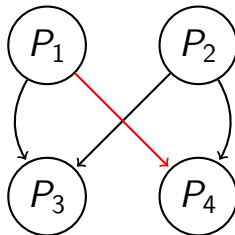
CoBegin P_1, P_2 ; **CoEnd**

CoBegin P_3, P_4 ; **CoEnd**

End

PPG and CoBegin/CoEnd

Let's enforce a precedence constraint on the PPG: We can add an arrow going from P_1 to P_4



Begin

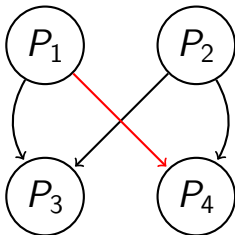
CoBegin P_1, P_2 ; **CoEnd**

CoBegin P_3, P_4 ; **CoEnd**

End

PPG and CoBegin/CoEnd

Let's enforce a precedence constraint on the PPG: We can add an arrow going from P_1 to P_4



Begin

CoBegin P_1, P_2 ; **CoEnd**

Fails if P_1 and P_4

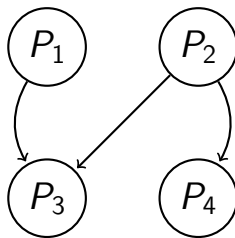
CoBegin P_3, P_4 ; **CoEnd**

need to cooperate!

End

PPG and CoBegin/CoEnd

Another try



CoBegin

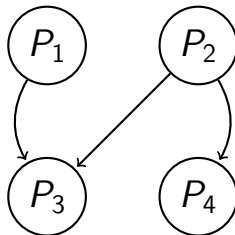
P_1

Begin $P_2; P_4$ **End**

CoEnd

$P_3;$

PPG and CoBegin/CoEnd



CoBegin

P_1

Begin $P_2; P_4$ **End**

Fails if P_3 and P_4

need to cooperate

CoEnd

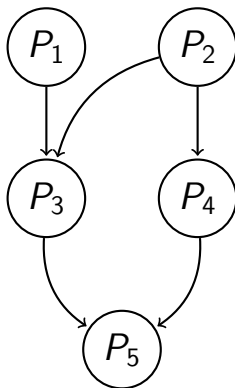
$P_3;$

CoBegin/CoEnd cannot represent arbitrary precedence constraints.

PPG with Semaphores (Waiting-style)

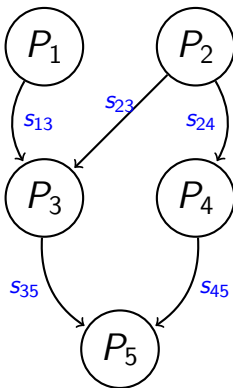
PPG and Synchronization (Semaphores)

Translate the following precedence graph into a code using semaphores:



PPG and Synchronization (Semaphores)

Brute force. Create as many semaphores as arrows:



$s_{13}, s_{23}, s_{24}, s_{35}, s_{45}$: Semaphore := 0; (5 semaphores)

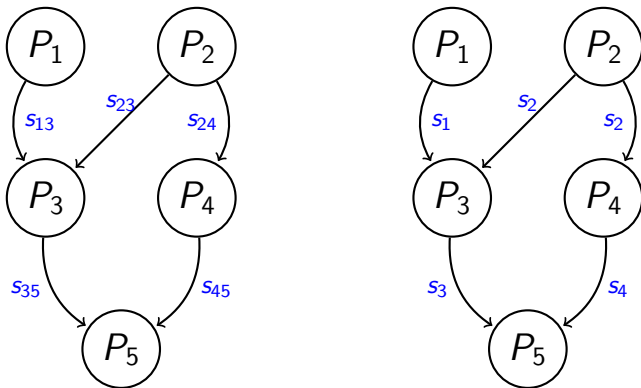
PPG and Synchronization (Semaphores)

Process 1	Process 2	Process 3	Process 4	Process 5
code ₁ ;	code ₂ ;	P(s ₁₃);	P(s ₂₄);	P(s ₄₅);
V(s ₁₃);	V(s ₂₃);	P(s ₂₃);	code ₄ ;	P(s ₃₅);
	V(s ₂₄);	code ₃ ;	V(s ₄₅);	code ₅ ;
		V(s ₃₅);		

s₁₃, s₂₃, s₂₄, s₃₅, s₄₅: Semaphore := 0; (5 semaphores)

PPG and Synchronization (Semaphores)

Optimization. Let us reduce the number of semaphores by renaming s_{xy} with s_x :



s_1, s_2, s_3, s_4 : Semaphore := 0; (4 semaphores)

PPG and Synchronization (Semaphores)

Process 1	Process 2	Process 3	Process 4	Process 5
code ₁ ;	code ₂ ;	P(s ₁);	P(s ₂);	P(s ₄);
V(s ₁);	V(s ₂);	P(s ₂);	code ₄ ;	P(s ₃);
	V(s ₂);	code ₃ ;	V(s ₄);	code ₅ ;
		V(s ₃);		

s₁, s₂, s₃, s₄: Semaphore := 0; (4 semaphores)

PPG and Synchronization (Semaphores)

Can you solve it with 3 semaphores?

PPG and Synchronization (Semaphores)

Process 1	Process 2	Process 3	Process 4	Process 5
code ₁ ;	code ₂ ;	P(s ₁);	P(s ₂);	P(s ₃);
V(s ₁);	V(s ₂);	P(s ₂);	code ₄ ;	P(s ₃);
	V(s ₂);	code ₃ ;	V(s ₃);	code ₅ ;
		V(s ₃);		

s₁, s₂, s₃: Semaphore := 0; (3 semaphores)

PPG and Synchronization (Semaphores)

Can we optimize the code further?

PPG and Synchronization (Semaphores)

Process 1	Process 2	Process 3	Process 4	Process 5
code ₁ ;	code ₂ ;	P(s ₁);	P(s ₂);	P(s ₃);
V(s ₁);	V(s ₂);	P(s ₂);	code ₄ ;	code ₅ ;
	V(s ₂);	code ₃ ;	V(s ₃);	
		V(s ₃);		

Semaphore $s_1 = 0$, $s_2 = 0$, $s_3 = -1$; (3 semaphores)

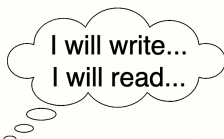
Classical Synchronization Problems

Classical Synchronization Problems

Readers & Writers Problem



Writers



Waiting Queue



Readers

Readers & Writers Problem

This problem consists of: (1) A set of reader threads $R = \{r_1, \dots, r_n\}$ & (2) a set of writer threads $W = \{w_1, \dots, w_m\}$. Both readers and writers share a data structure D (e.g., file, database, variable, ...) such that:

- Multiple readers $R' \subseteq R$ can read at the same time.
- Only one writer $w_i \in W$ can write at a time.
- If $\exists r \in R$ that is reading, new writers have to wait (no preemption).
- If $\exists w \in W$ that is writing, new writers and readers have to wait.
- If $\exists r \in R$ that is reading, new readers can start reading.

There exist three solutions: Readers have priority, Writers have priority, and starvation free solution.

Let's discuss the algorithm together...

Readers & Writers Problem ($|R| = |W| = 1$)

One reader $R = \{r\}$ and one writer $W = \{w\}$.

Semaphore $m = 1$;

r()

```
{  
    while(1)  
    {  
        acquire(m);  
        read();  
        release(m);  
        do_something_else();  
    }  
}
```

w()

```
{  
    while(1)  
    {  
        acquire(m);  
        write();  
        release(m);  
        do_something_else();  
    }  
}
```

Readers & Writers Problem ($|R| \geq |W| = m \geq 1$)

Multiple readers $R = \{r_1, \dots, r_n\}$ and multiple writers $W = \{w_1, \dots, w_m\}$.

1. Readers have priority. Let us discuss the solution where the Readers have priority.

Readers have priority means: when a new reader arrives and a writer is already waiting for previous readers to terminate, then the new reader **can** start reading with the other readers without waiting.

This solution is not a starvation-free solution.

e.g., If there is a steady stream of readers, writers may not get the chance to write.

Let's discuss the algorithm together...

Readers & Writers problem ($|R| \geq |W| = m \geq 1$)

Multiple readers $R = \{r_1, \dots, r_n\}$ and multiple writers $W = \{w_1, \dots, w_m\}$.

2. Writers have priority. This will be in **Lab 8**.

Writers have priority means: when a new reader arrives and a writer is already waiting for previous readers to terminate, then the new reader **cannot** start reading along with the other readers but has to wait.

AND

When there is waiting readers and writers during a writing, writers have the priority to be waken up over the readers at the end of the writing.

This solution is not a starvation-free solution.

i.e., If there is a steady stream of writers, readers may not get the chance to read.

Let's discuss the algorithm together...

- End.